

Rotman

INTRO TO R PROGRAMMING

R Tutorial (RSM358) – Session 4

October 1, 2026 Prepared by Jay Cao / [MDAL](#)

Website: <https://rmdal.github.io/r-intro-2026-fall/>



Rotman School of Management
UNIVERSITY OF TORONTO

A3 - Q14 Data Simulation, Any Questions?

- Q14 in Section 3.7: data simulation (collinearity setup)
- Read Section 3.3.3: #6 collinearity; #4, #5 outlier & leverage point

```
# simulation in Q14
set.seed(1)
x1 <- runif(100)
x2 <- 0.5 * x1 + rnorm(100) / 10
y <- 2 + 2 * x1 + 0.3 * x2 + rnorm(100)

# additional observation
x1 <- c(x1 , 0.1)
x2 <- c(x2 , 0.8)
y <- c(y, 6)
```

Binomial Logistic Regression

- let Y be a binary outcome variable (i.e., a binary categorical variable)
 - e.g. $Y = \{0, 1\} = \{fail, pass\}$, $Y = \{0, 1\} = \{down, up\}$, etc.
- Let $p = prob(Y = 1)$; $\frac{p}{1-p}$ is then the odds of being 1
 - The category of $Y = 0$ is a reference category
 - Reference category is relative as you can instead set $p = prob(Y = 0)$
- Binary logistic regression models the logit-transformed probability as a linear function of the predictor variables
 - Coefficients $(\beta_0 \dots \beta_k)$ are estimated using maximum likelihood method

$$logit(p) = \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k.$$

From Log Odds to Probability to Prediction

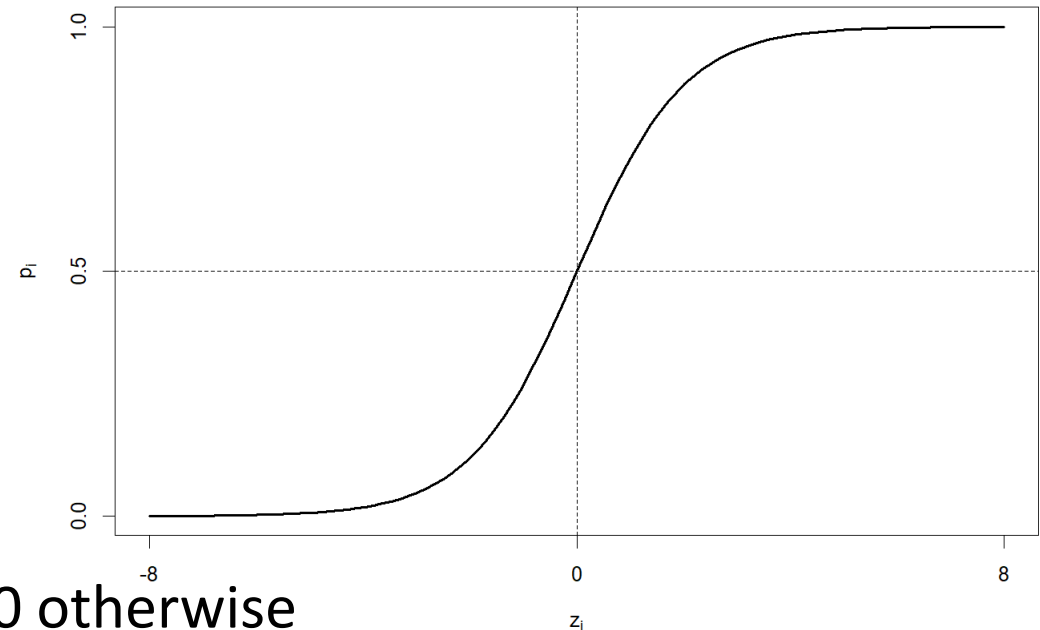
- Let $z_i = \text{logit}(p_i) = \log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_{1,i} + \dots + \beta_k x_{k,i}$.

- Then, $p_i = \frac{e^{z_i}}{1+e^{z_i}}$

- Note $0 < p_i < 1$

- Threshold prob t

- If the predicted $\hat{p}_i \geq t$, classify as 1, 0 otherwise
 - t is a hyper-parameter set by you (so you can tune it through validation)



Logistic Regression - Lab 4.7

- `my_model <- glm(formula = ..., data = ..., family=binomial)`
- `summary(my_model)`
- `predict(my_model, newdata = ..., type = "response")`
 - Set the argument `type = "response"` to get predicted probabilities, i.e., $P(Y = 1|X)$
 - Otherwise, `predict(my_model)` gives log odds (logit)
 - If the `newdata` argument is not supplied, the prediction is applied on the training data set
 - Use `contrast()` to find out which y category is set to 1.
- Construct confusing matrix
 - Convert probability prediction to binary prediction (cutoff prob.)
 - `table()`

A3 - Q14/a Prepare Data, Any Questions?

- Q14: load data, prepare a categorical feature, and the binary y

```
# Q14/a load the data
```

```
library(ISLR2)
```

```
auto <- Auto
```

```
# convert the origin column to factor type
```

```
auto$origin <- as.factor(auto$origin)
```

```
# prepare the binary variable y (mpg01) & convert the column type to factor
```

```
auto$mpg01 = ifelse(auto$mpg > median(auto$mpg), 1, 0)
```

```
auto$mpg01 <- as.factor(auto$mpg01)
```

Training & Test Set - Lab 4.7

Smarket

Year	Lag1	...	Direction
2001	0.381	...	Up
...	8	...	...
2005	...	...	Down

- Training and test set split
 - For time series data, need to respect the time when splitting the data
 - That is, train on early data, test on late data
 - Otherwise, randomly split data to train and test
- An example: a time series training & test set split from lab 4.7
 - Year and Direction are columns in the Smarket dataset
 - the Smarket data is “attached” so columns in Smarket are recognized as a variable (not a good practice in my opinion because...)

```
> train <- (Year < 2005) # train is a Boolean vector of length nrow(Smarket)
> Smarket.2005 <- Smarket[!train, ] # select all rows of Smarket where
> dim(Smarket.2005)                Year >= 2005, i.e., prepare test set X
[1] 252      9
> Direction.2005 <- Direction[!train] # select all data in Direction column
                                     where Year >= 2005, i.e., prepare test
                                     set y
```

Training & Test Set – A3 Q14/c

- Training and test set split
 - For time series data, need to respect the time when splitting the data
 - That is, train on early data, test on late data
 - Otherwise, randomly split data to train and test
 - The Auto data do have a model year column, but for simplicity we will treat the data as cross-sectional (i.e., not time series)

```
# Q14/c randomly split Auto dataset into training and test set
num_rows <- nrow(auto)
train_fraction <- 0.7
train_idx = sample(1:num_rows, size = round(num_rows * train_fraction))
train_data <- auto[train_idx, ]
test_data <- auto[-train_idx, ]
```

Confusion Matrix and Error Rate - Lab 4.7

```
> glm.fits <- glm(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume, # train/fit model on
  data = Smarket, family = binomial, subset = train      training set
)
> glm.probs <- predict(glm.fits, Smarket.2005,           # predict (prob) on test set;
  type = "response")                                   glm.probs is numerical vector.

> glm.pred <- rep("Down", 252)                          # predict (Up or Down) on test set;
> glm.pred[glm.probs > .5] <- "Up"                      glm.pred is a character vector.
> table(glm.pred, Direction.2005)                       # confusion matrix
      Direction.2005
glm.pred Down Up
      Down    77 97
      Up     34 44

> mean(glm.pred == Direction.2005) # accuracy: (TP + TN) / # of predictions
[1] 0.48
> mean(glm.pred != Direction.2005) # error rate = 1 - accuracy
[1] 0.52
#      = (FP + FN) / # of predictions
```

Naïve Bayes

- What is it?
 - See textbook 4.4.4
- How to fit it
 - See Lab 4.7.5
 - Install the e1071 library: `install.packages("e1071")`